FERHAOUI Yanisse 11909519

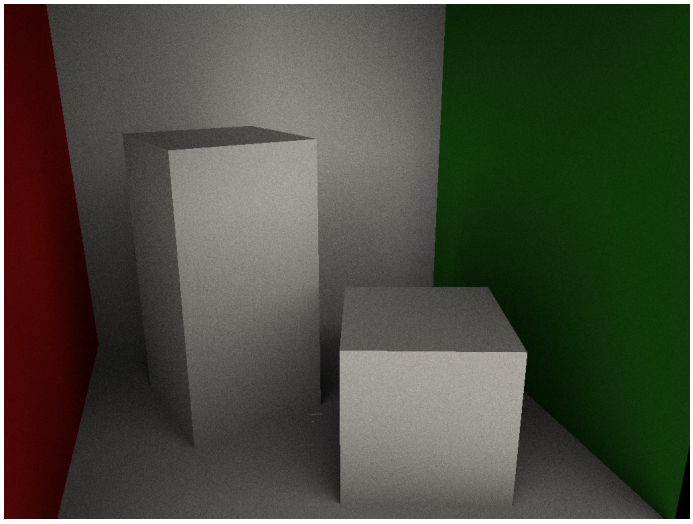
Rapport TP Lancer de rayons

Dans le fichier se trouve trois principales fonctions :

1. **Monte Carlo simple**

Cette fonction est celle qui permet de générer une direction aléatoire à l'aide de l'estimateur de Monte Carlo, la fonction **hemisphere** permet de générer le vecteur l à l'aide de 2 variables aléatoires u1 et u2 . Une fois la direction tirée, la fonction **isVisible** permet de calculer les ombres, et retourne vrai lorsque le rayon d'ombre touche un objet se trouvant entre le pixel testé et le ciel, faux sinon.

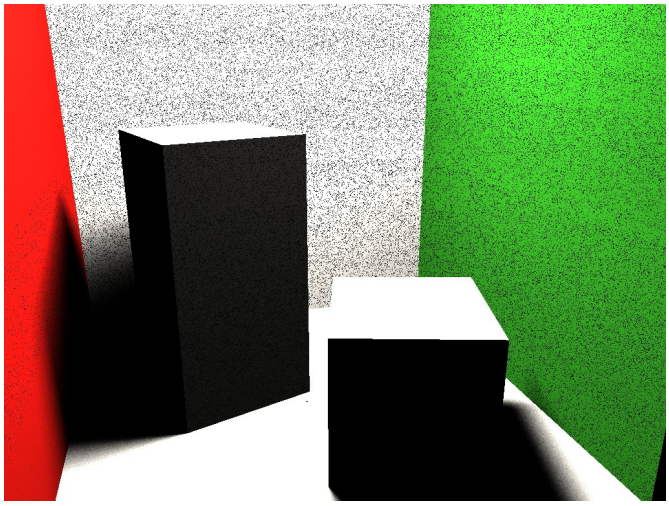
Avec N = 256 nous obtenons ceci :



2. **Monte Carlo**

Cette fonction fera la même chose que la précédente, mais cette fois-ci avec un éclairage direct et donc en utilisant des vraies lumières. Une structure *Source* a été créée à cet effet, contenant les points a, b, c ainsi que l'aire des triangles ayant une émission > 0. Un tableau de sources est rempli au début du programme principal. Pour le choix de la source (et ensuite du point q) la 1e solution de la documentation sur l'équation de rendu a été utilisée. La fonction **computeLe** permet de calculer l'émission Le(q, -l).

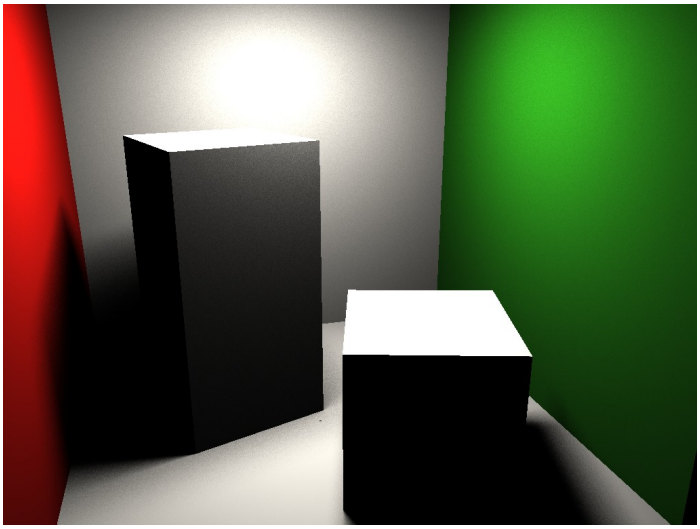
Avec N = 256 nous obtenons ceci :



3. Monte Carlo direct

La dernière étape consiste à rendre l'éclairage uniforme et plus efficace. Les changements réalisé par rapport à la dernière étape sont l'équation de rendu, ainsi que le choix du point q. Pour ce dernier la solution des coordonnées barycentriques du triangle source a été choisie.

Pour N = 256 nous obtenons ce résultat :



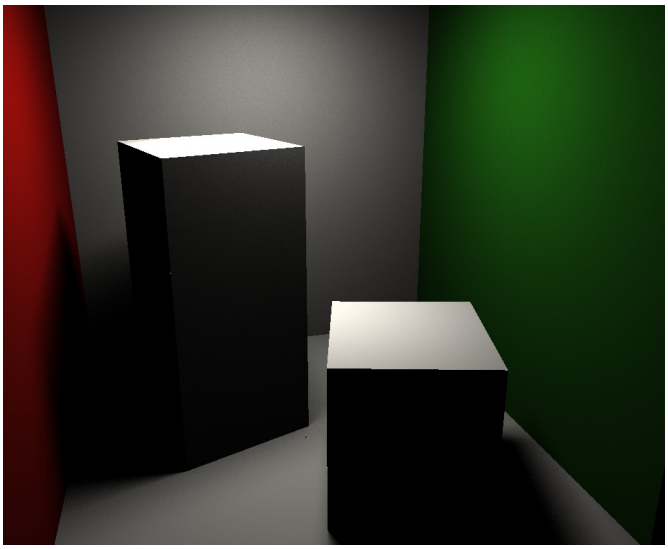
La scène finale se trouve à mon goût trop éclairée. En remplaçant la ligne

```
float pdf = source.aire / total * 1 / source.aire;
```

par :

```
float pdf = 1 / source.aire;
```

Le résultat est le suivant (toujours pour N = 256) :



Le code peut être compilé et lancé s’il est copié/collé dans un des fichiers tutos de gKit (il vient initialement de tuto_rayons.cpp).